

```
this._x--;  
}  
else if (action == "jump")  
{  
    this._y += this.yspeed;  
    this.yspeed++;  
}  
}
```

The code defined above is the generic code which uses a function called `update()` for updating an object. It is observed however that code written in this manner makes it difficult to add extra functions. To overcome this problem, the function can be made modular.

Making Code Modular

The above code can be modified by using a different type of function call which is shown below.

```
function update()  
{  
    action = action();  
}
```

5.4 Creating A Flash Application Using Function

In the above program segment, the code becomes much more simplified as all the conditional statements are omitted and only a function referrer is used for determining the next action. The parts of the code are broken into smaller segments for solving the problem with the generic function.

In this unit we have explained the various aspects of Functions, i.e. defining and calling functions, function statements and returning values from functions. Besides these, the concepts related to method formation and modular functions have also been discussed. Now let us create an effective application based on the above concepts. The detailed steps are given below.